

Capstone Project - MediAssist AI

Enterprise Healthcare Agentic AI Copilot

Project Overview

MediAssist AI is an enterprise-grade Healthcare AI Copilot that combines:

- Advanced RAG
- Multi-Agent AI
- Multimodal AI
- MCP PostgreSQL Tool Integration
- Dockerized Deployment
- Azure Cloud Hosting
- Enterprise Retrieval Systems

into one unified end-to-end Generative AI platform.

The system simulates a real-world hospital AI assistant where healthcare staff can:

- Chat with hospital knowledge bases
- Upload medical reports and prescriptions
- Query enterprise patient systems
- Retrieve contextual information
- Automate healthcare workflows
- Generate grounded recommendations

The project follows modern enterprise AI architecture patterns involving:

- agentic orchestration
 - MCP-based tool access
 - vector search pipelines
 - multi-agent workflows
 - production-grade evaluation systems
-

Business Problem

Hospitals and healthcare enterprises contain large amounts of fragmented data:

Data Type	Examples
Unstructured Data	SOPs, PDFs, policies
Multimodal Data	Prescriptions, reports
Structured Data	Patient systems, appointments
Operational Knowledge	Workflows, escalation procedures

Challenges include:

- Fragmented systems
 - Slow information retrieval
 - Manual operations
 - Knowledge silos
 - Lack of unified AI interface
 - Poor interoperability between enterprise systems
-

Project Objective

Build a Healthcare AI Copilot capable of:

- Conversational enterprise search
 - Agentic AI orchestration
 - Vision-based understanding
 - Structured + unstructured retrieval
 - Workflow automation
 - Enterprise tool integration using MCP
 - Containerized deployment on Azure
-

What Students Will Be Provided

Students are NOT expected to collect datasets manually.

The institute will provide all required enterprise-style data and schemas.

1. Unstructured Healthcare Documents

Students will receive healthcare enterprise documents such as:

Documents Provided

- Hospital SOPs
 - Clinical workflow PDFs
 - Insurance policy documents
 - Healthcare compliance guidelines
 - Discharge summaries
 - Operational manuals
 - Escalation workflows
 - Patient care guidelines
-

File Formats

Students will receive:

- PDF files
- DOCX documents
- TXT documents

These documents will be used for:

- RAG pipelines
 - Vector indexing
 - Semantic retrieval
 - Context grounding
-

2. Multimodal Data

Students will also receive healthcare image data.

Image Data Provided

- Scanned prescriptions
- Sample medical report images
- Lab report screenshots

These images will be used for:

- Vision-language understanding
- Information extraction

- Report understanding
 - Multimodal reasoning
-

3. PostgreSQL Database Access

Students will be provided with:

- PostgreSQL database connection details
- Database schema
- Table relationships
- Sample enterprise data

The database simulates an existing hospital operational system.

Enterprise Hospital Network

The healthcare platform operates across multiple hospital branches distributed across different cities.

Students should design solutions capable of supporting:

- Multi-branch healthcare operations
- Branch-level analytics
- Department-level reporting
- Cross-hospital patient insights
- Enterprise-wide retrieval and reasoning

This reflects how modern healthcare enterprises operate across multiple locations.

Students are NOT expected to build transactional healthcare systems.

Instead, they must integrate AI systems with enterprise databases using MCP architecture.

Database Schema Provided

Table	Purpose
Patients	Patient master records
Admissions	Hospital admission history
Appointments	Doctor appointments
Doctors	Doctor master data
Departments	Department master data
Billing	Billing and payments
Lab Results	Clinical test results
Prescriptions	Medication records
Insurance Policies	Insurance coverage information
Hospital Branches	Multi-location hospital network

Database details

Enterprise Data Scale

Students will work with an enterprise healthcare dataset consisting of approximately:

Table	Records
Patients	25,000
Admissions	50,000
Appointments	200,000
Billing	100,000
Prescriptions	250,000
Lab Results	500,000
Doctors	250
Departments	25
Insurance Policies	500
Hospital Branches	20

Total Records: ~1.1 Million Records

This better reflects enterprise-scale healthcare systems.

User Personas

Students should design workflows around realistic enterprise users.

Persona	Usage
Doctor	Retrieve patient summaries and reports
Hospital Administrator	Compliance and operational insights
Front Desk Staff	Appointment lookup and scheduling
Insurance Team	Billing and claims verification
Clinical Operations Team	Workflow and escalation management

Advanced Analytics Use Cases

The Healthcare AI Copilot should support not only operational retrieval but also analytical and decision-support use cases.

Examples:

- Which department had the highest admissions in 2025?
- Show appointment trends across hospital branches.
- Identify patients with abnormal lab results.
- Analyze billing by insurance provider.
- Compare admission rates across departments.
- Show top diagnoses by city.
- Analyze prescription patterns across specialties.
- Generate healthcare operational insights.

These use cases should leverage MCP tools,

PostgreSQL queries and agentic reasoning.

Core Architecture Principle

PostgreSQL Cannot Be Accessed Directly by AI Agents, Students MUST build:

MCP Server for PostgreSQL Access

The MCP server acts as a secure tool layer between AI agents and enterprise systems.

This reflects how enterprise AI systems increasingly expose tools and databases through standardized protocols like MCP.

Example MCP Tools

Students must expose tools such as:

```
get_patient_history()
```

```
get_appointments()
```

```
get_lab_results()
```

```
get_billing_summary()
```

```
get_patient_summary()
```

```
get_prescriptions()
```

```
get_admissions()
```

```
get_doctor_schedule()
```

```
search_patients()
```

```
search_lab_tests()
```

```
search_doctors()
```

```
get_insurance_details()
```

```
get_department_statistics()
```

The AI agents dynamically invoke these tools during workflow execution.

End-to-End User Workflow

Example User Query

User uploads a patient report and asks:

```
"Summarize this report, retrieve patient history,  
check upcoming appointments,  
and generate follow-up recommendations."
```

Additional Enterprise Queries

Operational Queries

- Show patient history for P10025
- Retrieve upcoming appointments
- Summarize latest lab reports

Analytical Queries

- Which department had the highest admissions this year?
- Compare appointment volume by hospital branch.
- Show patients with abnormal HbA1c results.
- Analyze insurance claims by provider.

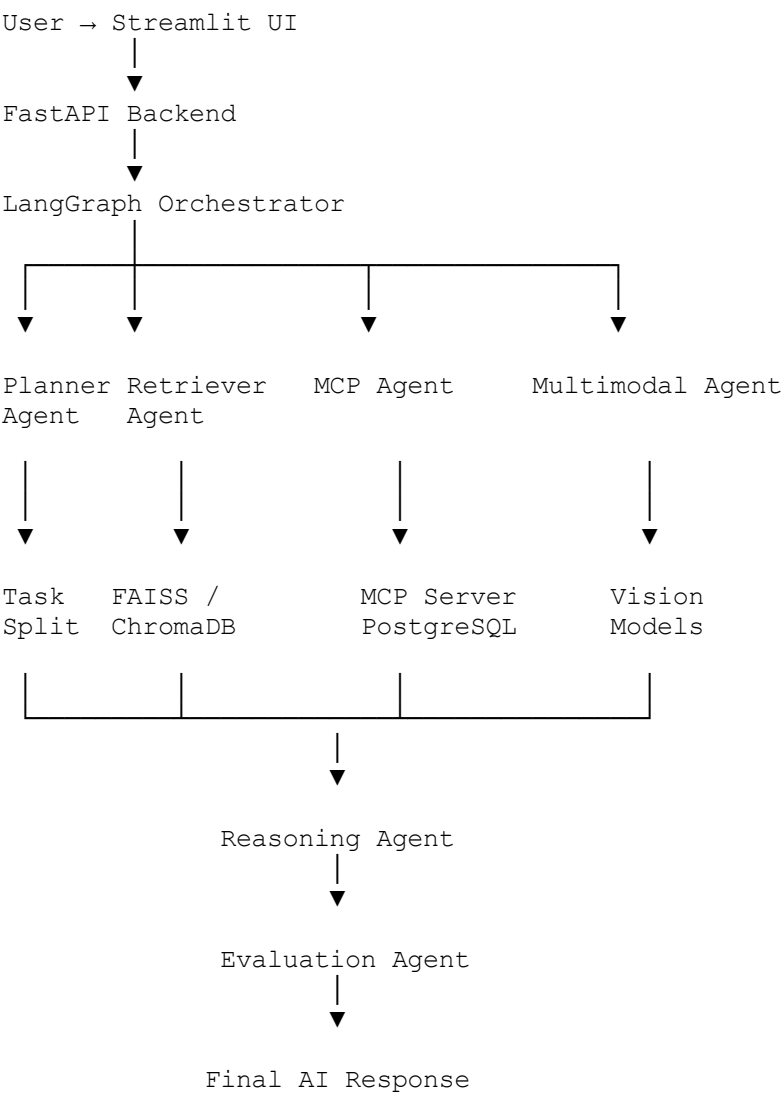
Multi-Agent Queries

- Retrieve patient history,
analyze latest lab results,
summarize prescriptions,
and generate recommendations.

Multimodal Queries

- Extract medicines from prescription image.
 - Summarize uploaded medical report.
 - Identify abnormalities from lab report image.
-

Complete System Flow



Detailed Workflow Explanation

Step 1 — User Interaction

User interacts through Streamlit UI.

The user can:

- Upload PDF reports
- Upload prescription images

- Ask natural language questions
-

Step 2 — Planner Agent

The Planner Agent:

- Understands the user request
- Breaks tasks into subtasks
- Decides which agents/tools are required

Example subtasks:

1. Analyze uploaded report
 2. Retrieve enterprise documents
 3. Query PostgreSQL tools
 4. Generate recommendations
-

Step 3 — Multimodal Agent

The Multimodal Agent performs:

- Vision understanding
- Medical report understanding
- Information extraction

Students will implement:

- BLIP
- LLaVA

Extracted information may include:

- Medicines
 - Diagnoses
 - Observations
 - Dates
-

Step 4 — Retriever Agent

The Retriever Agent:

- Queries vector databases
- Retrieves enterprise documents
- Performs semantic retrieval
- Returns grounded enterprise context

Students will implement:

- Chunking
- Embeddings
- Vector indexing
- Hybrid retrieval
- Metadata filtering

using:

- FAISS
OR
- ChromaDB

Vector databases remain a foundational component of enterprise RAG systems.

Step 5 — MCP Agent

The MCP Agent:

- Calls MCP tools
- Retrieves structured PostgreSQL data

Examples:

- Patient history
- Appointments
- Billing information
- Lab records

MCP-based enterprise integration is becoming a standard architecture pattern for tool orchestration and secure AI access layers.

Step 6 — Reasoning Agent

The Reasoning Agent combines:

Source	Information
Vector Search	Enterprise documents
MCP Output	PostgreSQL records
User Prompt	Current request
Vision Models	Medical report understanding

The agent generates:

- Clinical summaries
- Recommendations
- Grounded responses

Step 7 — Evaluation Agent

The Evaluation Agent validates:

- Hallucination risks
- Retrieval quality
- Grounding quality
- Confidence score
- Response correctness

Important Requirement

Students **MUST** implement:

LLM-as-a-Judge Evaluation

This is one of the most important components of the project.

Students should use an LLM to evaluate:

- answer quality
- faithfulness

- grounding
- hallucination detection
- response relevance

Evaluation and observability are increasingly becoming critical requirements in enterprise agentic AI systems.

Frontend Requirements (Streamlit)

Students must build a Streamlit application with:

Features

AI Chat Interface

- Conversational chatbot
 - Streaming responses
-

PDF Upload

Upload:

- SOPs
 - Policies
 - Reports
-

Image Upload

Upload:

- Prescriptions
 - Lab reports
 - Medical report images
-

Evaluation Dashboard

Display:

- Retrieval traces
 - Agent execution flow
 - Evaluation metrics
-

Technologies To Be Implemented

Layer	Technology
Frontend	Streamlit
Backend	FastAPI
LLM Framework	LangChain
Agent Framework	LangGraph
LLM	OpenAI / Groq
Embeddings	Hugging Face
Vector Search	FAISS / ChromaDB
Vision Models	BLIP / LLaVA
MCP	MCP Python SDK
Database Integration	PostgreSQL
Evaluation	LLM-as-a-Judge
Containerization	Docker
Cloud Hosting	Azure/AWS

Project Modules

Module 1 — Data Ingestion Pipeline

Students will implement:

- PDF ingestion
 - Image ingestion
 - Chunking strategy
 - Embedding generation
 - Metadata creation
-

Module 2 — Enterprise RAG Pipeline

Students will implement:

- Vector database indexing
- Semantic retrieval
- Hybrid retrieval
- Citation-based grounding
- Metadata filtering

using:

- FAISS
 - OR
 - ChromaDB
-

Module 3 — Multi-Agent AI System

Students will build:

- Planner Agent
- Retriever Agent
- MCP Agent
- Multimodal Agent
- Reasoning Agent
- Evaluation Agent

using LangGraph.

Agentic orchestration using LangGraph and MCP is increasingly becoming the dominant enterprise AI architecture pattern.

Module 4 — MCP Server

Students will implement:

- MCP server
- PostgreSQL connectors
- MCP tools

- Tool invocation workflows

This is one of the strongest enterprise-level components in the project.

Module 5 — Multimodal AI

Students will implement:

- Vision-language processing
 - Medical report understanding
 - Image-based reasoning
-

Module 6 — Evaluation & Observability

Students will implement:

- LLM-as-a-Judge evaluation
 - Hallucination detection
 - Retrieval evaluation
 - Workflow monitoring
-

Module 7 — Guardrails & Safety

Students should implement safeguards for:

- Prompt injection detection
- Sensitive information protection
- Unsafe recommendation detection
- Hallucination prevention

- Tool misuse prevention

The system should ensure that generated responses remain grounded in retrieved data.

Module 8 — Dockerization

Students will create:

- Dockerfile
 - Docker Compose setup
-

Module 9 — Azure Deployment

Students will deploy:

- Streamlit application
 - FastAPI backend
 - Docker containers
 - Azure Container Apps hosting
-

Non-Functional Requirements

Students should ensure:

- Modular architecture
 - API-based communication
 - Containerized deployment
 - Scalable retrieval pipeline
 - Agent observability
 - Error handling
 - Secure credential management
 - Config-driven architecture
 - Logging and tracing
-

Deliverables

Students Must Submit

1. GitHub Repository

Repository must include:

- Source code
 - Docker setup
 - API code
 - Technical documentation, testing for edge cases (template)
 - Deployment scripts
-

2. Architecture Documentation

Documentation must include:

- System architecture
 - Agent workflow
 - MCP architecture
 - Deployment flow
-

3. Working Demo

Students must demonstrate:

- Enterprise RAG
 - MCP integration
 - PostgreSQL tool access
 - Multimodal uploads
 - Multi-agent orchestration
-

4. Azure Deployment

The application must be deployed on Azure.

5. Evaluation Report

Must include:

- Retrieval quality metrics
 - Hallucination analysis
 - Latency analysis
 - Agent performance evaluation
 - LLM-as-a-Judge evaluation results
-

Skills Covered

Area	Skills
Generative AI	RAG, Prompting, Agents
AI Engineering	LangChain, LangGraph
Multimodal AI	Vision Models
Enterprise AI	MCP, Tool Calling
Cloud	Azure Hosting
Backend Engineering	FastAPI APIs
DevOps	Docker
LLMOps	Evaluation & Monitoring

Learning Outcomes

By completing this capstone, students will understand:

- Enterprise AI architecture
 - Production-grade RAG systems
 - Agentic AI orchestration
 - MCP-based enterprise integration
 - Structured + unstructured retrieval
 - Vector database pipelines
 - Multimodal AI systems
 - Containerized AI deployment
 - Enterprise observability and evaluation
-

Industry Relevance

This capstone closely mirrors modern enterprise AI systems built using:

- Agentic workflows
- MCP-based tool orchestration
- Vector search systems
- Multi-agent AI
- Enterprise retrieval architectures
- Evaluation-driven AI systems

Enterprise AI systems are increasingly evolving from isolated chatbot implementations toward orchestrated, tool-aware, multi-agent platforms.